Carnegie Mellon University
School of Computer Science · Executive Education

Module 7: Capstone Project

Agent Dobby

Autonomous AI-Powered Swing Trading Pipeline

Antonysamy Joseph · leenuslee@gmail.com
September 2026

Table of Contents

1. Project Title
2. Problem and User
3. System Goal and Scope
4. Final System Architecture
5. Design Evolution Across the Program
6. Implementation Overview
7. Evaluation and Results
8. Safety and Reliability Considerations
9. Limitations and Next Steps
10. Public GitHub Repository

SECTION 1

Project Title

Agent Dobby: Autonomous AI-Powered Swing Trading Pipeline

Agent Dobby is a multi-agent, AI-driven automated trading system implemented in Python. The system integrates large language model (LLM) reasoning, technical indicator analysis, Retrieval-Augmented Generation (RAG) for news sentiment, and brokerage execution into a cohesive autonomous pipeline designed to support personal swing trading activities.

SECTION 2

Problem and User

Target User

The intended user is an individual retail investor engaged in swing trading — a strategy that holds positions from several days to a few weeks. The user is technically proficient but is not available to monitor markets continuously during trading hours.

Problem Statement

Retail swing traders face three compounding challenges:

- **Information overload:** Thousands of news articles, earnings reports, and technical signals are published daily. Manually synthesizing these into actionable decisions is time-consuming and error-prone.
- **Emotional decision-making:** Human traders frequently deviate from their own defined entry/exit rules under market pressure, leading to suboptimal outcomes.
- **Monitoring gaps:** Swing trading setups trigger at unpredictable times during market hours. Consistent, timely evaluation requires continuous observation that most individuals cannot sustain.

Agent Dobby addresses these challenges by providing an autonomous system that continuously monitors a curated watchlist, evaluates technical setups against real-time market data, incorporates news-based sentiment through RAG, and executes orders against a brokerage account — all according to pre-approved, back-tested trading strategies.

Why This Problem Matters

Automated trading tools are largely inaccessible to individual retail investors without deep financial or engineering resources. Building a personal, extensible Agentic AI platform addresses this gap and simultaneously provides hands-on experience with modern autonomous systems architecture — multi-agent coordination, reasoning loops, vector retrieval, and safe automated execution.

SECTION 3

System Goal and Scope

Primary Goal

Build an intelligent, autonomous swing trading pipeline that demonstrates modern Agentic AI architecture while serving as a practical personal investment assistant.

System Capabilities

- Monitor a watchlist of stocks and evaluate entry conditions using named, back-tested trading setups.
- Evaluate open holdings against exit conditions and execute sell orders when triggered.
- Run historical backtests of any named setup against real market data with full performance metrics.
- Ingest and summarize financial news into a local vector database for semantic search.
- Expose a conversational chat interface for interrogating and controlling the system in natural language.
- Support a mock/simulation mode with synthetic tickers for safe, reproducible testing.

Successful Performance Criteria

Criterion	Target
Multi-agent coordination	Portfolio manager orchestrates research, evaluation, and execution sub-agents autonomously
Technical indicator evaluation	50+ condition types evaluated deterministically per bar
News sentiment retrieval	RAG pipeline ingests and embeds articles 3× daily; top-5 semantic results retrieved per query
Backtest accuracy	Chronological bar-by-bar simulation with stop-loss, take-profit, and position sizing
Trade execution	Market orders placed and confirmed via Alpaca API with fill polling
Chat interface	18 tools across 5 domains respond correctly to natural language queries
Simulation mode	Day-by-day historical replay with no lookahead, results persisted to database

Scope Boundaries

- **In scope:** Multi-agent AI platform, historical market data, technical indicators, paper trading, chat interface, portfolio monitoring, backtesting, local deployment.
- **Out of scope (current release):** Live account trading with real money, real-time streaming data, high-frequency strategies, cloud deployment, multi-broker support.

SECTION 4

Final System Architecture

Layered Architecture Overview

The system is organized into five layers: External Services, Tools, Agents, Scheduling/Chat, and Data. Each layer communicates only with adjacent layers, preserving loose coupling.

Agent Descriptions

Agent	Type	Responsibility
PortfolioManagerAgent	Supervisor (plain Python)	Orchestrates the full run: triggers holdings evaluation concurrently, runs research pipeline, invokes technical evaluator, executes buys sequentially, persists run summary.
ResearchAgent	LangGraph ReAct	Retrieves news from ChromaDB, fetches live price, synthesizes a BUY/SELL/HOLD/WAIT recommendation with reasoning.
BuyTechEvaluatorAgent	Deterministic Python	Evaluates all entry conditions (AND logic) against the latest OHLCV bar. Returns BUY or WAIT with per-condition breakdown.
CurrentHoldingsEvaluatorAgent	Deterministic Python	Evaluates exit conditions (OR logic) + stop-loss/take-profit for all open holdings. Executes sell orders sequentially after parallel evaluation.

Agent	Type	Responsibility
ChatAgent	LangGraph ReAct	Conversational interface with 18 tools. Persists conversation via MemorySaver. Supports candlestick charts, backtest results, portfolio queries.

Tree-of-Thought Buy Decision Flow

The buy decision pipeline within PortfolioManagerAgent implements a two-level Breadth-First Search (BFS) reasoning flow:

- **Level 1 — Sentiment Filter (ResearchAgent):** For each (ticker, setup) pair on the watchlist, the ResearchAgent queries ChromaDB for recent news summaries and returns a sentiment-weighted BUY/WAIT recommendation. Only BUY-rated tickers advance.
- **Pruning after Level 1:** Portfolio guardrails (available cash ≥ $100, allocation limits) are applied to narrow the candidate set.
- **Level 2 — Technical Evaluation (BuyTechEvaluatorAgent):** All shortlisted (ticker, setup) pairs are evaluated in parallel. All entry conditions must be MET (AND logic).
- **Final Execution:** Passing pairs are executed sequentially via Alpaca market orders. Position size = risk_percent × available_cash / price.

RAG Pipeline

Yahoo Finance RSS feeds are ingested three times daily. Each article is summarized using Claude Haiku (or a local Ollama model), deduplicated by content hash, embedded using the all-MiniLM-L6-v2 model (384-dimensional local embeddings), and stored in ChromaDB. Semantic search retrieves the top-5 most relevant summaries per ticker/query pair, which are injected into the ResearchAgent's context.

Data Layer

Store	Technology	Contents
Relational DB	PostgreSQL	portfolio_accounts, portfolio_holdings, portfolio_transactions, close_trade_references, backtest_runs, watchlist, pm_agent_runs
Setup Store	SQLite	Named trading setup definitions (JSON) — portable, fast, no joins required
Vector Store	ChromaDB (local)	News article summaries with ticker/sentiment metadata, 384-dim embeddings
OHLCV Cache	CSV files	Daily OHLCV + 30+ computed indicators per ticker, refreshed every 23 hours

Reasoning Loops and Memory

- **Short-term memory:** LangGraph MemorySaver maintains conversation context across turns within a thread_id. Large array fields (candle_data, trades, holdings) are trimmed after each response to prevent

context window growth.

• **Long-term memory:** All trade executions, backtest runs, portfolio manager run summaries, and watchlist configurations are persisted to PostgreSQL, enabling system restart and historical replay.

• **Lazy graph initialization:** LangGraph graphs are built on first invocation, not at module import, avoiding connection overhead at startup.

SECTION 5

Design Evolution Across the Program

The system evolved through six modules, each adding a distinct architectural capability. The table below summarizes the key refinement at each stage.

Module	Theme	Key Addition / Refinement
1	Initial Architecture (LTP.pdf)	Defined the multi-agent vision: SupervisorAgent, PortfolioManagerAgent, BackTestingAgent. Established technology choices: Python, LangGraph, LangChain, PostgreSQL, Alpaca. Identified two goals: learn Agentic AI and build a practical trading assistant.
2	Reasoning, Memory & Tools (LTPDesignDetails.pdf)	Formalized ReAct as the core reasoning pattern. Defined short-term memory (LangGraph MemorySaver) and long-term memory (PostgreSQL). Identified four tool categories: news/market data, OHLCV extraction, technical indicators, and brokerage API.
3	Multi-Agent Structure (LTPMultiAgentDesign.pdf)	Expanded to seven agents: ChatSupportAgent, BackTesterAgent, PortfolioManagerAgent, CurrentHoldingEvaluatorAgent, BuyResearchAgent, BuyEvaluatorAgent, TradeExecutorAgent. Defined synchronous and asynchronous agent communication patterns.
4	RAG Integration (LTPRAGDesign.pdf)	Added a dedicated RAG service layer. ResearchAgent ingests Yahoo Finance RSS, summarizes with LLM, chunks if needed, stores in local ChromaDB. Defined semantic search retrieval injected into buy/sell LLM context.
5	Tree-of-Thought (LTPToTDesign.pdf)	Incorporated two-level BFS decision tree in PortfolioManagerAgent. Level 1: news-sentiment filter. Level 2: technical indicator setup evaluation. Each level followed by guardrail-based pruning before advancing.
6	Guardrails (LTPGuardrails.pdf)	Defined nine guardrail levels from code review through system failure recovery. Introduced paper-trading-first policy, per-agent system prompts, back-tested-setups-only constraint, portfolio allocation limits, and a planned human-in-the-loop approval queue.
Final	Implemented System (agentdobby.md)	Delivered working system: 50+ condition registry, Backtester with Sharpe/drawdown metrics, mock simulation runner, chat API with 18 tools, context trimming, already_formatted flag pattern, FIFO trade matching, parallel research with sequential execution.

Most Significant Refinements

• **Technical evaluation decoupled from LLM:** Early designs relied on an LLM + MCP tool call to evaluate technical conditions. This was slow and expensive. The final design implements all 50+ conditions as deterministic Python factories in condition_registry.py, enabling local-model compatibility and eliminating per-evaluation API costs.

• **Deterministic supervisor, LLM sub-agents:** PortfolioManagerAgent was redesigned as plain Python (not LangGraph). Only ResearchAgent and ChatAgent use LangGraph ReAct loops. This makes the critical trade orchestration path testable, predictable, and fast.

• **Sequential execution, parallel evaluation:** Holdings evaluation runs in a background thread while the buy pipeline proceeds. Research runs in parallel (unless Ollama is used). Order execution is always sequential to prevent cash over-spend.

• **Mock/simulation mode:** The MOCK_STOCKS_ENABLED flag and pm_simulation_runner enable reproducible historical testing without live API calls — a critical safety addition before paper trading validation.

SECTION 6

Implementation Overview

Technology Stack

Layer	Technology	Role
Language	Python 3.12+	Primary implementation language
Agent Framework	LangGraph + LangChain	ReAct reasoning loops, MemorySaver, tool bindings
LLM (primary)	Claude Sonnet 4.6 (Anthropic)	Agent reasoning, trade evaluation, chat responses
LLM (summarization)	Claude Haiku (Anthropic)	News article summarization, setup parsing from natural language
LLM (local alt.)	Ollama (qwen3.5:9b, gemma4)	Local model support; sequential execution enforced (max_workers=1)
Market Data	Alpha Vantage API	Daily OHLCV + 23h local CSV cache; 30+ indicators via ta library
Brokerage	Alpaca API (alpaca-py)	Paper trading: market order submission, fill polling, account queries
News Source	Yahoo Finance RSS	News article ingestion for RAG pipeline
Embeddings	all-MiniLM-L6-v2 (local)	384-dim sentence embeddings, no API call required
Vector Store	ChromaDB (local)	News summary storage and semantic retrieval
Relational DB	PostgreSQL + SQLAlchemy ORM	Portfolio, trades, backtests, pm_agent_runs, watchlist
Setup Store	SQLite (SetupStore)	Portable JSON setup definitions, keyed by name and UUID
Scheduler	APScheduler	Market-hours cron jobs for PM agent and news ingestion

Layer	Technology	Role
Chat API	FastAPI (uvicorn)	REST endpoints: POST /login, POST /whatnext
Chat UI	React + Vite	AgentLee UI — candlestick charts, backtest panels, plain text

Key Modules

• **tools/alphavantage/alphavantage_data.py:** Single source of truth for OHLCV and all technical indicators (SMA/EMA families, RSI, MACD, Bollinger Bands, ATR, ADX, OBV, Stochastic, derived crossover signals, candlestick patterns).

• **tools/trade_setup/condition_registry.py:** 50+ parameterized condition factories using lambda/closure pattern. Conditions are evaluated row-by-row at backtest and live evaluation time with no LLM involvement.

• **agents/portfolio_manager_agent.py:** Supervisor orchestrator. Manages concurrent holdings evaluation and sequential buy execution. All cash adjustments occur within the same SQLAlchemy session as the trade record.

• **backtest/backtester.py:** Bar-by-bar chronological simulation with position sizing (equity_pct, cash_pct, fixed_qty), stop-loss, take-profit, and metric computation (total return, win rate, Sharpe ratio, max drawdown).

• **chat_support/chat_agent.py:** LangGraph ReAct agent with 18 tools and MemorySaver. already_formatted flag short-circuits the second LLM pass for structured JSON responses. Priority map ensures the most informative result wins in parallel tool calls.

• **jobs/pm_simulation_runner.py:** Advances MOCK_SIMULATION_DATE one business day at a time, calling PortfolioManagerAgent for each date. Persists daily results to pm_agent_runs for review via chat interface.

Configuration

All configuration is managed via environment variables loaded through config.py. MODEL_PROVIDER selects the LLM backend (anthropic / ollama / qwen / gemma). MOCK_STOCKS_ENABLED switches the system to synthetic-data mode. ALPACA_PAPER controls paper vs. live brokerage routing. WARMUP_DAYS=250 ensures sufficient history for SMA-200 computation.

SECTION 7

Evaluation and Results

Evaluation Approach

The system was evaluated through three complementary methods: historical simulation using mock tickers, paper trading via Alpaca, and backtesting against real market data.

Mock Simulation

Five synthetic tickers (DOBBY, OWALA, MINI, LUCAS, PILLOW) with pre-generated OHLCV CSV files and ChromaDB news articles enable fully deterministic replay. The pm_simulation_runner advances MOCK_SIMULATION_DATE one business day at a time, preventing lookahead bias. Results are persisted to pm_agent_runs and reviewable via the chat interface.

- End-to-end pipeline verified: research → evaluation → order simulation → trade persistence → holdings update.
- Cash balance consistency verified: buy decrements and sell increments confirmed within the same SQLAlchemy transaction.
- FIFO matching confirmed: multiple open lots for the same ticker matched in chronological order on close.

Backtest Evaluation

The Backtester class was validated against the RSI_MACD_TREND and EMA_PULLBACK example setups over 365-day windows on real tickers.

Metric	Description
Total Return	Final value vs. initial cash; verified against manual trade log
Win Rate	Percentage of profitable closed trades
Average Win / Loss	Mean P&L; on winning and losing trades respectively
Sharpe Ratio	Risk-adjusted return (annualized, zero risk-free rate)
Max Drawdown	Largest peak-to-trough equity decline during test window
Trade Count	Total number of completed round-trip trades

Technical Condition Evaluation

Each condition in condition_registry.py was independently verified by comparing its output on known OHLCV sequences against manual calculations. The BuyTechEvaluatorAgent returns a per-condition breakdown (label, type, params, met: true/false) enabling transparent inspection of every decision.

Chat Interface Coverage

Tool File	Tools	Test Coverage
chat_agent_tools.py	get_candlebar_data, get_recommendation	Candlestick data rendered in React UI; recommendation text validated
backtest_tools.py	run_backtest, list_backtest_runs, get_backtest_result	Full backtest triggered via chat; results panel rendered
setup_tools.py	save_trade_setup, list_trade_setups, get_trade_setup	Natural-language setup parsed by Claude Haiku; conditions validated against registry
watchlist_tools.py	add/update/list watchlist entries	CRUD operations confirmed via chat and PostgreSQL
portfolio_tools.py	create account, holdings, trades	Account creation, holdings listing, open positions verified
pm_run_tools.py	list/get pm_run_results	Simulation run summaries retrieved and displayed

Limitations Observed During Evaluation

- Local Ollama models (qwen3.5:9b, gemma4) require sequential research execution and 90-second per-ticker timeouts to prevent deadlocks on limited hardware.
- Stale MemorySaver threads may answer from prior context without calling tools; documented workaround is to start a new thread_id for clean state.
- Alpha Vantage free-tier rate limits require careful scheduling; the 23-hour CSV cache mitigates most API call pressure.

SECTION 8

Safety and Reliability Considerations

Multi-Level Guardrail System

The system implements a nine-level guardrail framework designed to be progressively tightened before any real-money deployment.

Level	Guardrail	Implementation Status
0	Thorough code review of all generated code	Implemented — all modules reviewed line-by-line
1	Model selection and switching	Implemented — MODEL_PROVIDER env variable; abstraction allows hot-swap across anthropic / ollama / qwen / gemma

Level	Guardrail	Implementation Status
2	Paper trading before live accounts	Implemented — ALPACA_PAPER=true routes all orders to Alpaca paper account
3	System-level prompts per agent	Implemented — each agent carries a distinct system prompt directing tool-first behavior
4	Research agent evaluation with multiple models	Implemented — mock mode skips LLM for deterministic testing; multi-model comparison planned
5a	Back-tested setups only	Implemented — watchlist entries reference named setups validated through Backtester
5b	Technical evaluation agent verification	Implemented — condition_registry validated independently; deterministic Python replaces LLM for indicator eval
6	Portfolio-level allocation constraints	Implemented — risk_percent × available_cash position sizing; minimum cash threshold ($100)
7	Human-in-the-loop before trade execution	Planned — approval queue and notification mechanism before live deployment
8	Monitoring and alerts	Partial — scheduler run summaries logged to pm_agent_runs; full LangSmith integration planned
9	System failure recovery	Partial — PostgreSQL persistence enables restart; Alpaca sync tool planned for live accounts

Implemented Fallback Logic

- **Per-ticker research timeout (90s):** Each ResearchAgent call is wrapped in a ThreadPoolExecutor future with a 90-second timeout. On timeout, the ticker defaults to WAIT and the pipeline continues without blocking.
- **Sequential order execution:** All buy and sell order submissions are executed sequentially, never in parallel, to prevent cash over-commitment or double-spending.
- **Holdings evaluation in background thread:** CurrentHoldingsEvaluatorAgent runs concurrently with the buy pipeline but is awaited with a 5-minute timeout before the run is finalized.
- **Mock executor:** MOCK_STOCKS_ENABLED routes all orders through simulate_fill() — Alpaca is never called for synthetic tickers, preventing accidental real orders during testing.
- **Cash consistency via SQLAlchemy session:** save_trade() adjusts portfolio_accounts.cash within the same session as the trade record, ensuring atomicity.
- **FIFO trade matching:** Close trades are matched to opening lots in chronological order via close_trade_references, providing an auditable and consistent cost-basis record.

Human Oversight Model

The current system operates in a supervised-automation mode: the portfolio manager scheduler runs autonomously during market hours, but all actions are logged to pm_agent_runs and reviewable via the chat

interface. The planned Level 7 guardrail will introduce a notification-and-approval queue that pauses before each trade execution, giving the operator final authority over every order before any real-money deployment.

SECTION 9

Limitations and Next Steps

Current Limitations

Category	Limitation
Deployment	Runs entirely on a local development machine. No cloud redundancy, no automated failover if the host goes offline during market hours.
Live trading	Restricted to Alpaca paper trading. Live account support requires completion of Level 7 (human approval queue) and extensive paper trading validation.
Local model quality	Ollama models (qwen3.5:9b, gemma4) struggle with structured tool-call responses. Anthropic API is required for reliable chat and research agent performance, incurring API costs.
No real-time data	Alpha Vantage free tier provides end-of-day OHLCV only. Intraday price movements are not evaluated; the system is suited only for daily-bar swing trading.
Context window growth	MemorySaver threads accumulate history; the trimming heuristic mitigates but does not eliminate stale-context risk over long sessions.
Backtesting scope	Backtests use end-of-day data only. Slippage, partial fills, and market impact are not modeled, so backtest results may overstate live performance.
News latency	RSS ingestion runs 3× daily. Breaking news between ingestion cycles is not captured, potentially missing sentiment shifts within the trading day.

Realistic Next Steps

- **Level 7 guardrail (human-in-the-loop):** Implement an approval queue that notifies via email/SMS and waits for confirmation before submitting any live order.
- **AWS deployment:** Migrate to EC2/ECS with RDS PostgreSQL for continuous availability. Add automated backup and restore procedures.
- **LangSmith monitoring integration:** Add structured tracing for all LangGraph agent runs to enable performance analysis and anomaly detection.
- **Live account support:** Enable ALPACA_PAPER=false after sustained paper trading validation, with portfolio-level hard stops and daily loss limits.
- **Intraday data tier:** Integrate a paid Alpha Vantage or Polygon.io tier for intraday OHLCV to support shorter-duration setups.
- **Risk Manager Agent:** Dedicated agent that enforces macro-level constraints (sector concentration, correlation limits, volatility-based position sizing) across the full portfolio.
- **Earnings Analysis Agent:** Agent that monitors earnings calendars and adjusts position recommendations during earnings blackout windows.

- **Reinforcement learning for strategy optimization:** Use historical simulation outcomes to tune setup parameters and allocation weights.
- **Multi-broker support:** Abstract the TradeExecutorAgent to support additional brokers (Interactive Brokers, TD Ameritrade) beyond Alpaca.
- **Voice interface:** Add a voice-driven command layer on top of the ChatAgent for hands-free portfolio queries.

SECTION 10

Public GitHub Repository

Repository

The project source code is maintained in a public GitHub repository to allow technical review of the implementation.

Repository URL: *https://github.com/antonysamys/agent-dobby* (to be made public upon submission)

Repository Contents

Path / File	Description
README.md	Project overview, architecture diagram, setup instructions, environment variable reference, and usage examples
agents/	All agent implementations: portfolio_manager_agent.py, research_agent.py, buy_tech_evaluator_agent.py, current_holdings_evaluator_agent.py, chat_support/
tools/	AlphaVantage data fetch + indicator computation, Alpaca trade executor, RAG pipeline (rag_yahoo/), condition registry, SetupStore, mock data and executor
backtest/	Backtester class, CLI runner, example setup factories, seed script
jobs/	APScheduler jobs: portfolio_manager_scheduler.py, rag_data_ingester.py, pm_simulation_runner.py
tools/db/	SQLAlchemy ORM models and data access layer for all PostgreSQL tables
agentlee-ui/	React + Vite chat interface with candlestick charts and backtest result panels
tools/mock_data/	Synthetic ticker CSV files (DOBBY, OWALA, MINI, LUCAS, PILLOW) and mock news generator
config.py	Centralized environment variable loading and derived settings
.env.example	Template for all required environment variables

Running the System

```
# Initialize database python -m tools.db.init_db # Start news ingestion scheduler (7AM, 12PM, 8PM ET Mon-Fri) python -m jobs.rag_data_ingester # Start portfolio manager scheduler (9:30AM + hourly 10AM-3PM Mon-Fri) python -m jobs.portfolio_manager_scheduler # Start chat API uvicorn chat_support.chat_api:app --reload --port 8800 # Run simulation over historical date range python3 -m jobs.pm_simulation_runner --account <uuid> --start 2026-01-01 --end 2026-06-30 # Run a backtest from CLI python -m backtest.backtest_runner NVDA RSI_MACD_TREND --days 365
```